Lecture 25.10: kNN Implementation in Scikit-Learn

📌 Topic Overview

In this lecture, Shraddha ma'am writes the complete Scikit-Learn code for **KNeighborsClassifier**, emphasizing why **feature scaling is strictly mandatory** before passing data into kNN.

⚠️ Why Scaling is Non-Negotiable for kNN

kNN relies directly on Euclidean distance:

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

If one feature has values around 100,000 and another has values around 2, the unscaled distance will be 99.99% driven by the first feature alone. Without `StandardScaler`, kNN degrades into a single-feature model!

💻 Python Implementation

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

def train_knn_with_proper_scaling(n_neighbors=5):
    """
    Demonstrates training KNeighborsClassifier with mandatory StandardScaler.
    Feature scaling is strictly mandatory for kNN because distances are
    distorted by unscaled features.
    """
    # 1. Dataset with different feature scales
    X, y = make_classification(n_samples=400, n_features=4, random_state=42)
    # Artificially blow up scale of feature 0
    X[:, 0] = X[:, 0] * 1000

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
```

```
    # 2. Scaling (MANDATORY for kNN)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled  = scaler.transform(X_test)

    # 3. Fit kNN Model
    knn = KNeighborsClassifier(n_neighbors=n_neighbors)
    knn.fit(X_train_scaled, y_train)

    # 4. Predict & Score
    acc = knn.score(X_test_scaled, y_test)

    return knn, acc

# --- Function Call & Testing ---
model, test_acc = train_knn_with_proper_scaling(n_neighbors=5)
print(f"KNeighborsClassifier (k=5) Test Accuracy: {test_acc*100:.2f}%")
```

🔑 Key Takeaways

1. Always apply `StandardScaler` before fitting `KNeighborsClassifier`.
2. Tune `n_neighbors` (standard default is 5).
3. Ensure no data leakage: `fit_transform` on train, `transform` on test.